

MODULE 5

Claude Code, Hooks, Skills & Subagents

Agentic architecture, deterministic hooks, SKILL.md, subagent delegation, Agent SDK, and GitHub CI/CD integration

Domain 4 — 22% of CCA-F Exam

Claude Certified Architect (CCA-F) | Professional Enterprise Architect | May 2026

What You Will Master in This Module

- Claude Code: dual-model, context sources, auto-compaction, 9 built-in tools
- CLAUDE.md: probabilistic user context vs deterministic system prompt — architecture impact
- Hooks system: 6 lifecycle events, 4 handler types — deterministic governance
- Hook patterns: block dangerous commands, auto-lint, Slack notifications, SIEM
- Agent Skills (SKILL.md): complete frontmatter schema, triggering, distribution
- Subagents: isolated contexts, custom agent definitions, structured output design
- Multi-agent patterns: orchestrator-worker, pipeline, router, fan-out/fan-in
- Claude Agent SDK: TypeScript programmatic agent execution for CI/CD
- GitHub Actions: automated PR review, test generation, code quality gates

5.1 Claude Code Architecture

Claude Code is Anthropic's terminal-based agentic platform. More accurately: a general computer automation framework — anything achievable via terminal is within scope.

Primary model	Claude Sonnet 4.6 for main agentic loop: planning, tool selection, synthesis
Secondary model	Claude Haiku 4.5 for lightweight sub-tasks: file routing, quick lookups
Context window	200K tokens: system prompt + tools + CLAUDE.md + session history + tool results + buffer
Auto-compaction	Triggers at ~75-92% capacity. Summarizes old turns. Session continuity preserved but CLAUDE.md changes
Search strategy	Ripgrep (grep-based) — NOT vector/RAG. Claude decides what to search iteratively. Better for codebases
Built-in tools (9)	Read · Write · Bash · Grep · Glob · WebSearch · Task (spawn subagent) · TodoRead · TodoWrite
Config directory	.claude/ — contains settings.json, commands/, agents/, skills/, hooks/
Memory system	File-based markdown (no vector DB). Auto-generated, injected into CLAUDE.md. Inspectable, editable,

Critical: CLAUDE.md is User Context, NOT System Prompt

CLAUDE.md is delivered probabilistically as user context. Claude may override it with its own judgment. For mandatory rules, use Hooks — they execute deterministic code that cannot be bypassed by the model.

5.2 Hooks System — Deterministic Governance

Hooks fire at lifecycle events and run deterministic code — the model cannot override them. This makes hooks the correct choice for security controls, compliance gates, and mandatory quality checks.

Event	Trigger	Control	Use Cases
PreToolUse	BEFORE any tool	Exit 0=allow, 1=warn, 2=BLOCK	Block dangerous commands, enforce gates, compliance checks
PostToolUse	AFTER tool completes	Modify output before Claude sees it	Security lint, audit logging, quality validation
UserPromptSubmit	On user message	Modify or block prompt	Context injection, input sanitization, logging
Stop	Session ends	Informational only	Slack notifications, session summary, cleanup
SubagentStop	Subagent completes	Informational only	Aggregate results, validate subagent outputs
Notification	Claude requests attention	Informational only	Desktop alerts, mobile push, Slack pings

Handler Types: 4 Implementation Options

Type	Description	Example
Command (shell)	Shell script via stdin JSON. Exit code controls. Fastest option for simple yes/no patterns	Block dangerous file paths
Prompt (LLM eval)	Single-turn Claude evaluation for nuanced policy judgment. Does this change expose credentials?	Does this change expose credentials?

Agent (subagent)	Full subagent with tool access for complex verification. Can run tests, suites	Can run tests, suites coverage drops
Webhook	HTTP POST to external system. For SIEM, change manager to audit SIEM	Everything Agent Does, BLOCKED to audit SIEM

PreToolUse Hook — Block Dangerous Commands

```
#!/usr/bin/env bash # .claude/hooks/pre-bash.sh - exit 0=allow, 1=warn, 2=BLOCK CMD=$(cat | python3 -c 'import sys,json; print(json.load(sys.stdin)["tool_input"]["command"])') BLOCK_PATTERNS=('rm -rf /' 'DROP TABLE' 'chmod 777' 'curl.*|.bash' '> /etc') for p in "${BLOCK_PATTERNS[@]}"; do if echo "$CMD" | grep -qiE "$p"; then echo "BLOCKED: matches '$p'" >&2; exit 2 fi done if echo "$CMD" | grep -qiE '(production|prod-db|--force)'; then echo "WARNING: references production environment" >&2; exit 1 fi exit 0
```

5.3 Agent Skills (SKILL.md)

Skills package domain expertise into reusable instruction modules. They prevent repeated copy-pasting and ensure consistent execution of common tasks.

```
--- name: api-endpoint-generator description: > Use when creating a new REST API endpoint, adding a route, building a controller. TRIGGER: 'add endpoint', 'create route', 'new API', 'add controller'. DO NOT USE FOR: modifying existing endpoints, GraphQL, gRPC, WebSocket. allowed-tools: [Read, Write, Bash, Grep] # Least privilege disallowed-tools: [WebSearch] # Explicit block scripts: post-create: | npm run lint:fix npm run type-check model: claude-sonnet-4-6 effort: medium context: fork # Isolated subagent - no context pollution maxTurns: 25 --- # API Endpoint Generator ## Phase 1: Understand the Codebase 1. Read src/routes/index.ts - understand routing pattern 2. Read one existing controller - understand structure 3. Read one existing service - understand business logic patterns ## Phase 2: Create Files (in this order) 1. src/models/{name}.model.ts - TypeScript interfaces 2. src/repositories/{name}.repository.ts - Prisma data access 3. src/services/{name}.service.ts - business logic, Result 4. src/controllers/{name}.controller.ts - HTTP handler, Zod validation 5. Register route in src/routes/index.ts 6. src/tests/{name}.test.ts - minimum 3 test cases ## Final Report Files created: [list with line counts] Route: [METHOD /path] Tests: [count] cases Assumptions: [decisions made without explicit instruction]
```

5.4 Subagents, Multi-agent Patterns & Agent SDK

Context isolation	Sub-agent has ZERO access to parent history. Only what's in the Task prompt is available.
Tool restriction	Define allowed-tools per agent. Security reviewer: Read+Grep only. Test runner: Bash+Read only.
Structured output	Instruct sub-agents to return JSON. Parent agent parses and synthesizes results.
maxTurns	Always set. Short tasks: 10-20 turns. Complex analysis: 30-50 turns. Prevents runaway agents.
context: fork	Runs in isolated subagent context. context:current runs inline. Use fork for long/heavy skills.

Multi-agent Patterns

Pattern	Description	Example
Orchestrator-Worker	Main decomposes → workers run in parallel → main aggregates	100 files → 10 workers of 10
Pipeline/Chain	Output of each stage feeds next → sequential refinement	Draft → Review → Edit → Ship

Router	Classifier routes to specialists → reduces context pollution	Ticket → billing/tech/shipping agent
Fan-out/Fan-in	N agents evaluate same problem → main synthesizes	5 architecture perspectives → unified review

Agent SDK — TypeScript CI/CD Integration

```
import { ClaudeAgent } from '@anthropic-ai/claude-agent-sdk'; async function reviewPR(diff: string)
{ const agent = new ClaudeAgent({ model: 'claude-sonnet-4-6', tools: ['Read', 'Grep', 'Glob'], // No
Write – read-only review maxTurns: 15, onToolUse: async (tool, input, run) => {
console.log(`[${tool}]`, input); return run(); // Execute + can modify result } }); const result =
await agent.run( `Review this PR for security, correctness, and test coverage:\n${diff}\n Return
JSON: {severity:'high|medium|low', findings:[...], approved:bool}` ); return
JSON.parse(result.content[0].text); }
```